

STAT4911_Weather_Model

Srinika Kalavala

2026-05-01

```
### Libraries
library(dplyr)
library(terra)
library(tidyr)
library(glmnet)
library(stringr)
library(ggplot2)
```

```
### Processing the weather data
process_weather <- function(r) {

  pt <- data.frame(x = -95.375, y = 29.75)
  pt_vect <- vect(pt, geom = c("x", "y"), crs = crs(r))

  df <- terra::extract(r, pt_vect)

  # If df has ID column (common), remove it
  if ("ID" %in% colnames(df)) {
    df <- df[, -1, drop = FALSE]
  }

  df_t <- as.data.frame(t(df))

  df_t$Datetime <- time(r)
  df_t$Variable <- names(r)

  return(df_t)
}
```

Loading + Merging Datasets

```
weather_data_may_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-may2022.grib")
weather_data_june_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-june2022.grib")
weather_data_july_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-july2022.grib")
weather_data_august_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-august2022.grib")
weather_data_september_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-september2022.grib")
weather_data_october_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-october2022.grib")
weather_data_november_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-november2022.grib")
weather_data_december_2022 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2022/data-december2022.grib")
```

```
weather_data_january_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-january2023.grib")
weather_data_february_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-february2023.grib")
weather_data_march_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-march2023.grib")
weather_data_april_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-april2023.grib")
weather_data_may_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-may2023.grib")
weather_data_june_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-june2023.grib")
weather_data_july_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-july2023.grib")
weather_data_august_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-august2023.grib")
weather_data_september_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-september2023.grib")
weather_data_october_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-october2023.grib")
weather_data_november_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-november2023.grib")
weather_data_december_2023 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2023/data-december2023.grib")
```

```
weather_data_january_2024 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2024/data-january2024.grib")
weather_data_february_2024 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2024/data-february2024.grib")
weather_data_march_2024 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2024/data-march2024.grib")
weather_data_april_2024 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2024/data-april2024.grib")
weather_data_may_2024 <- rast("/Users/srinika/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/Weather Data/2024/data-may2024.grib")
```

```
weather_all <- bind_rows(
  process_weather(weather_data_may_2022),
  process_weather(weather_data_june_2022),
  process_weather(weather_data_july_2022),
  process_weather(weather_data_august_2022),
  process_weather(weather_data_september_2022),
  process_weather(weather_data_october_2022),
  process_weather(weather_data_november_2022),
  process_weather(weather_data_december_2022),
  process_weather(weather_data_january_2023),
  process_weather(weather_data_february_2023),
  process_weather(weather_data_march_2023),
  process_weather(weather_data_april_2023),
  process_weather(weather_data_may_2023),
  process_weather(weather_data_june_2023),
  process_weather(weather_data_july_2023),
  process_weather(weather_data_august_2023),
  process_weather(weather_data_september_2023),
  process_weather(weather_data_october_2023),
  process_weather(weather_data_november_2023),
  process_weather(weather_data_december_2023),
  process_weather(weather_data_january_2024),
  process_weather(weather_data_february_2024),
  process_weather(weather_data_march_2024),
  process_weather(weather_data_april_2024),
  process_weather(weather_data_may_2024)
)
```

```

weather_all <- weather_all %>%
  group_by(Datetime, Variable) %>%
  summarise(V1 = mean(V1, na.rm = TRUE), .groups = "drop")

weather_wide <- weather_all %>%
  pivot_wider(
    id_cols = Datetime,
    names_from = Variable,
    values_from = V1
  )

```

```

weather_wide <- weather_wide %>%
  mutate(
    Datetime = as.POSIXct(Datetime, tz = "UTC"),
    Datetime = round(Datetime, "hour")
  )

```

```

var_dict <- data.frame(
  column = names(weather_wide)[names(weather_wide) != "Datetime"],
  grib_name = names(weather_wide)[names(weather_wide) != "Datetime"]
)

```

Load the dataset

```

battery_dataset = read.csv("~/Desktop/OSU/Junior Year/Spring/STAT 4911/Battery Project/battery_dataset_20260327.csv")

```

```

battery_dataset_all <- battery_dataset %>%
  mutate(
    Datetime = as.POSIXct(Datetime, format = "%m/%d/%y %H:%M", tz = "UTC"),
    Datetime = round(Datetime, "hour")
  ) %>%
  filter(
    Datetime >= as.POSIXct("2022-05-01", tz = "UTC") &
    Datetime < as.POSIXct("2023-01-01", tz = "UTC")
  )

```

Merging the datasets

```

merged_data <- battery_dataset_all %>%
  left_join(weather_wide, by = "Datetime")
dim(merged_data)

```

```

## [1] 10562    118

```

```

model_data <- merged_data %>%
  select(-Date, -AccountNumber, -UnitOfMeasure, -PriceUnits) %>%
  select(where(is.numeric))
model_data$Datetime <- merged_data$Datetime

model_data <- model_data %>%
  mutate(
    hour = as.numeric(format(Datetime, "%H")),
    day = as.numeric(format(Datetime, "%d"))
  )

# Had a datatype issue so I changes SettlementPointPrice to numeric
merged_data$SettlementPointPrice <- as.numeric(merged_data$SettlementPointPrice)

# Removed V6 because almost every row is missing
model_data <- model_data %>%
  select(-any_of("V6"))

# Cleaning the NA's
model_data <- model_data %>%
  select(where(~ mean(is.na(.)) < 0.5)) %>%
  na.omit()

```

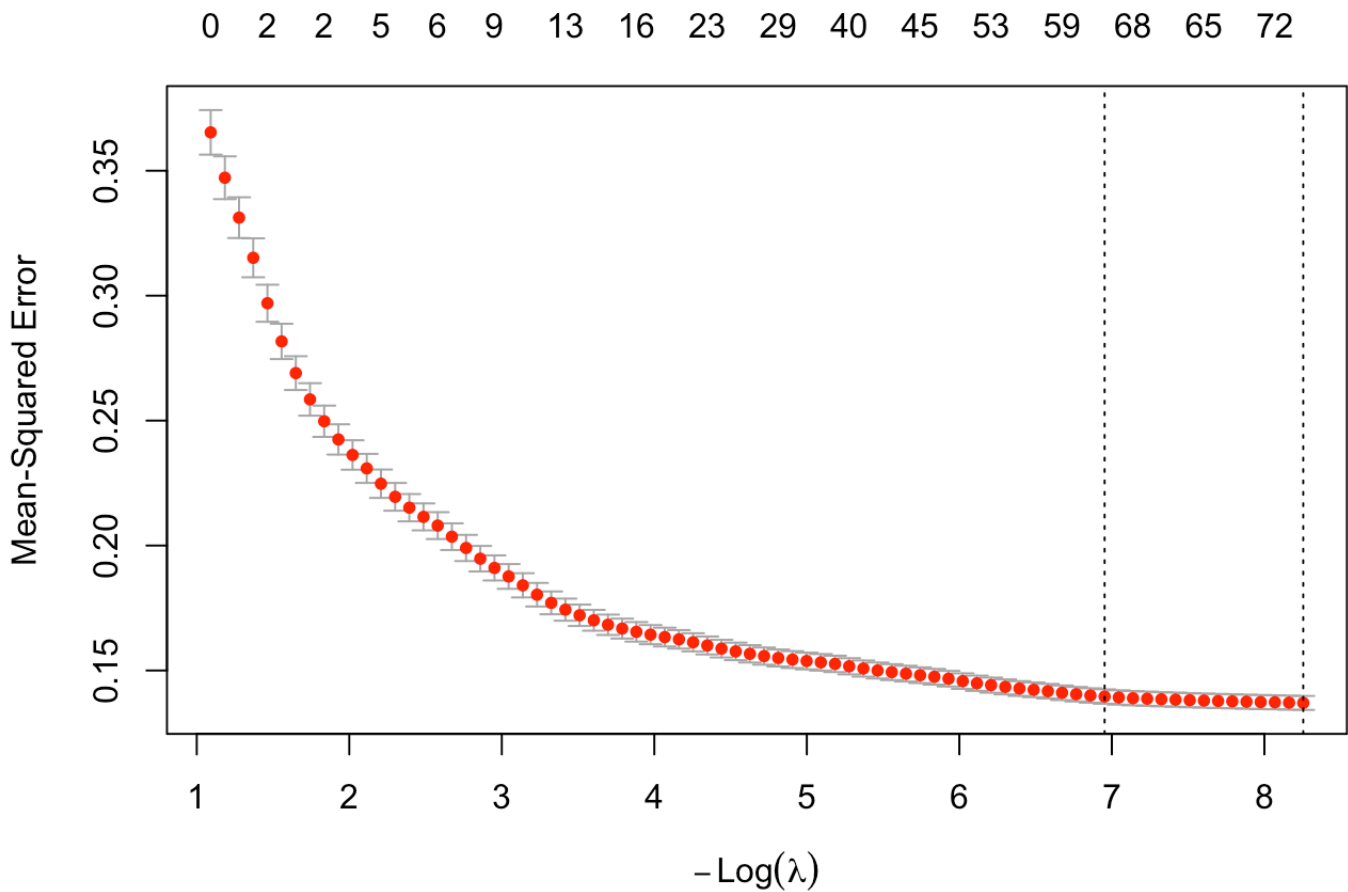
```

### Modeling
# Decided to do a LASSO because we have a high-dimensional dataset
X <- model.matrix(SettlementPointPrice ~ . -Datetime, data = model_data)[,-1]
y <- log(model_data$SettlementPointPrice)

# Train/Test
set.seed(4911)
train_idx <- sample(1:nrow(X), 0.8 * nrow(X))
X_scaled <- scale(X)
X_scaled <- X_scaled[, colSums(is.na(X_scaled)) == 0]
X_train <- X_scaled[train_idx, ]
y_train <- y[train_idx]
X_test <- X_scaled[-train_idx, ]
y_test <- y[-train_idx]

# Training model
lasso_model <- cv.glmnet(X_train, y_train, alpha = 1)
plot(lasso_model)

```



```
# Performance metrics
pred_log <- predict(lasso_model, s = "lambda.min", newx = X_test)
pred <- exp(pred_log) / 1000 # <--- DIVIDE BY 1000 HERE
y_test_original <- exp(y_test) / 1000 # <--- DIVIDE BY 1000 HERE

mse <- mean((pred - y_test_original)^2)
mse
```

```
## [1] 0.007585488
```

```
# Get the coefficient matrix from the LASSO model
coef_matrix <- coef(lasso_model, s = "lambda.1se")

print("---- sum ----")
```

```
## [1] "---- sum ----"
```

```
sum(coef_matrix != 0)
```

```
## [1] 66
```

```
coef_df <- data.frame(  
  Feature = rownames(coef_matrix),  
  Coefficient = as.numeric(coef_matrix)  
)  
  
# Remove intercept  
coef_df <- coef_df %>%  
  filter(Feature != "(Intercept)")  
  
# Remove zeros  
coef_df <- coef_df %>%  
  filter(Coefficient != 0)  
  
# Sort  
coef_df <- coef_df %>%  
  arrange(desc(Coefficient))  
  
top_10_positive <- head(coef_df, 10)  
top_10_negative <- tail(coef_df, 10)  
  
print("Top 10 Positive Drivers")
```

```
## [1] "Top 10 Positive Drivers"
```

```
print(top_10_positive)
```

```

##
Feature
## 1                                `SFC (Ground or water surface); To
p solar radiation [W*s/m^2]`
## 2                                `SFC (Ground or water surface);
Evaporation (of water) [m]`
## 3                                `SFC (Ground or water surfac
e); u-component of wind [m/s]`
## 4                                `SFC (Ground or water surface); Near IR albed
o for diffuse radiation [-]`
## 5                                `SFC (Ground or water surface); Surface thermal ra
diation downwards [W*s/m^2]`
## 6    `7-28[cm] DBLY (layer between 2 depths below land surface); Deep soil tmp/soil
tmp lev2(from 930804) [C]`
## 7    `28-100[cm] DBLY (layer between 2 depths below land surface); Clim deep soil tm
p/soil tmp lev3(930804) [C]`
## 8                                `SFC (Ground or water surf
ace); Relative humidity [%]`
## 9                                `SFC (Ground or water s
urface); Cloud Ice [kg/m^2]`
## 10                               `SFC (Ground or water surface); 10 m
etre u wind component [m/s]`
##      Coefficient
## 1      1.04511880
## 2      0.41122355
## 3      0.29203758
## 4      0.18810174
## 5      0.17920878
## 6      0.16989199
## 7      0.15953373
## 8      0.13315826
## 9      0.12443443
## 10     0.09918421

```

```
print("Top 10 Negative Drivers")
```

```
## [1] "Top 10 Negative Drivers"
```

```
print(top_10_negative)
```



```

##
Feature
## 56                                     `SFC (Ground or water surface); Prec
ipitable water [kg/m^2]^
## 57  `28-100[cm] DBLY (layer between 2 depths below land surface); Volumetric soil
water layer 3 [m^3/m^3]^
## 58                                     `SFC (Ground or water surface); Top ther
mal radiation [W*s/m^2]^
## 59                                     `SFC (Ground or water surface); Max 2m temp since previ
ous post-processing [C]^
## 60  `0-7[cm] DBLY (layer between 2 depths below land surface); Volumetric soil
water layer 1 [m^3/m^3]^
## 61                                     `SFC (Ground or water surface); 10 metre
v wind component [m/s]^
## 62                                     `SFC (Ground or water surface); Instantaneous su
rface Heat Flux [W/m^2]^
## 63  `0-7[cm] DBLY (layer between 2 depths below land surface); Surf.temp/soil temp
lev 1 (from 930804) [C]^
## 64                                     `SFC (Ground or water surface); Precip
itation rate [kg/m^2/s]^
## 65                                     `SFC (Ground or water surface); Min 2m temp since previ
ous post-processing [C]^
##      Coefficient
## 56 -0.02986686
## 57 -0.03449873
## 58 -0.04117244
## 59 -0.06709069
## 60 -0.08402853
## 61 -0.08917561
## 62 -0.10941276
## 63 -0.13029501
## 64 -0.19540672
## 65 -0.37714517

```

```
print("--- Top 10 Positive Drivers ---")
```

```
## [1] "--- Top 10 Positive Drivers ---"
```

```
print(top_10_positive)
```

```

##
Feature
## 1                                     `SFC (Ground or water surface); To
p solar radiation [W*s/m^2]`
## 2                                     `SFC (Ground or water surface);
Evaporation (of water) [m]`
## 3                                     `SFC (Ground or water surfac
e); u-component of wind [m/s]`
## 4                                     `SFC (Ground or water surface); Near IR albed
o for diffuse radiation [-]`
## 5                                     `SFC (Ground or water surface); Surface thermal ra
diation downwards [W*s/m^2]`
## 6   `7-28[cm] DBLY (layer between 2 depths below land surface); Deep soil tmp/soil
tmp lev2(from 930804) [C]`
## 7   `28-100[cm] DBLY (layer between 2 depths below land surface); Clim deep soil tm
p/soil tmp lev3(930804) [C]`
## 8                                     `SFC (Ground or water surf
ace); Relative humidity [%]`
## 9                                     `SFC (Ground or water s
urface); Cloud Ice [kg/m^2]`
## 10                                    `SFC (Ground or water surface); 10 m
etre u wind component [m/s]`
##   Coefficient
## 1   1.04511880
## 2   0.41122355
## 3   0.29203758
## 4   0.18810174
## 5   0.17920878
## 6   0.16989199
## 7   0.15953373
## 8   0.13315826
## 9   0.12443443
## 10  0.09918421

```

```
print("--- Top 10 Negative Drivers ---")
```

```
## [1] "--- Top 10 Negative Drivers ---"
```

```
print(top_10_negative)
```

```

##
Feature
## 56                                     `SFC (Ground or water surface); Prec
ipitable water [kg/m^2]^
## 57  `28-100[cm] DBLY (layer between 2 depths below land surface); Volumetric soil
water layer 3 [m^3/m^3]^
## 58                                     `SFC (Ground or water surface); Top ther
mal radiation [W*s/m^2]^
## 59                                     `SFC (Ground or water surface); Max 2m temp since previ
ous post-processing [C]^
## 60  `0-7[cm] DBLY (layer between 2 depths below land surface); Volumetric soil
water layer 1 [m^3/m^3]^
## 61                                     `SFC (Ground or water surface); 10 metre
v wind component [m/s]^
## 62                                     `SFC (Ground or water surface); Instantaneous su
rface Heat Flux [W/m^2]^
## 63  `0-7[cm] DBLY (layer between 2 depths below land surface); Surf.temp/soil temp
lev 1 (from 930804) [C]^
## 64                                     `SFC (Ground or water surface); Precip
itation rate [kg/m^2/s]^
## 65                                     `SFC (Ground or water surface); Min 2m temp since previ
ous post-processing [C]^
##      Coefficient
## 56 -0.02986686
## 57 -0.03449873
## 58 -0.04117244
## 59 -0.06709069
## 60 -0.08402853
## 61 -0.08917561
## 62 -0.10941276
## 63 -0.13029501
## 64 -0.19540672
## 65 -0.37714517

```

```

df_pos <- top_10_positive %>%
  mutate(
    Feature_Clean = str_wrap(Feature, width = 50),
    Feature_Clean = reorder(Feature_Clean, Coefficient)
  )

df_neg <- top_10_negative %>%
  mutate(
    Feature_Clean = str_wrap(Feature, width = 50),
    Feature_Clean = reorder(Feature_Clean, -Coefficient)
  )

# Top 10 Positive Drivers (Price Increasers)
plot_positive <- ggplot(df_pos, aes(x = Coefficient, y = Feature_Clean)) +
  geom_bar(stat = "identity", fill = "red", alpha = 0.8) +
  theme_minimal() +
  labs(
    title = "Top 10 Positive Price Drivers",
    subtitle = "Factors associated with higher electricity prices",
    x = "Coefficient Magnitude",
    y = NULL
  ) +
  theme(
    axis.text.y = element_text(size = 9, lineheight = 0.7),
    plot.title = element_text(face = "bold", size = 12, color = "red")
  )

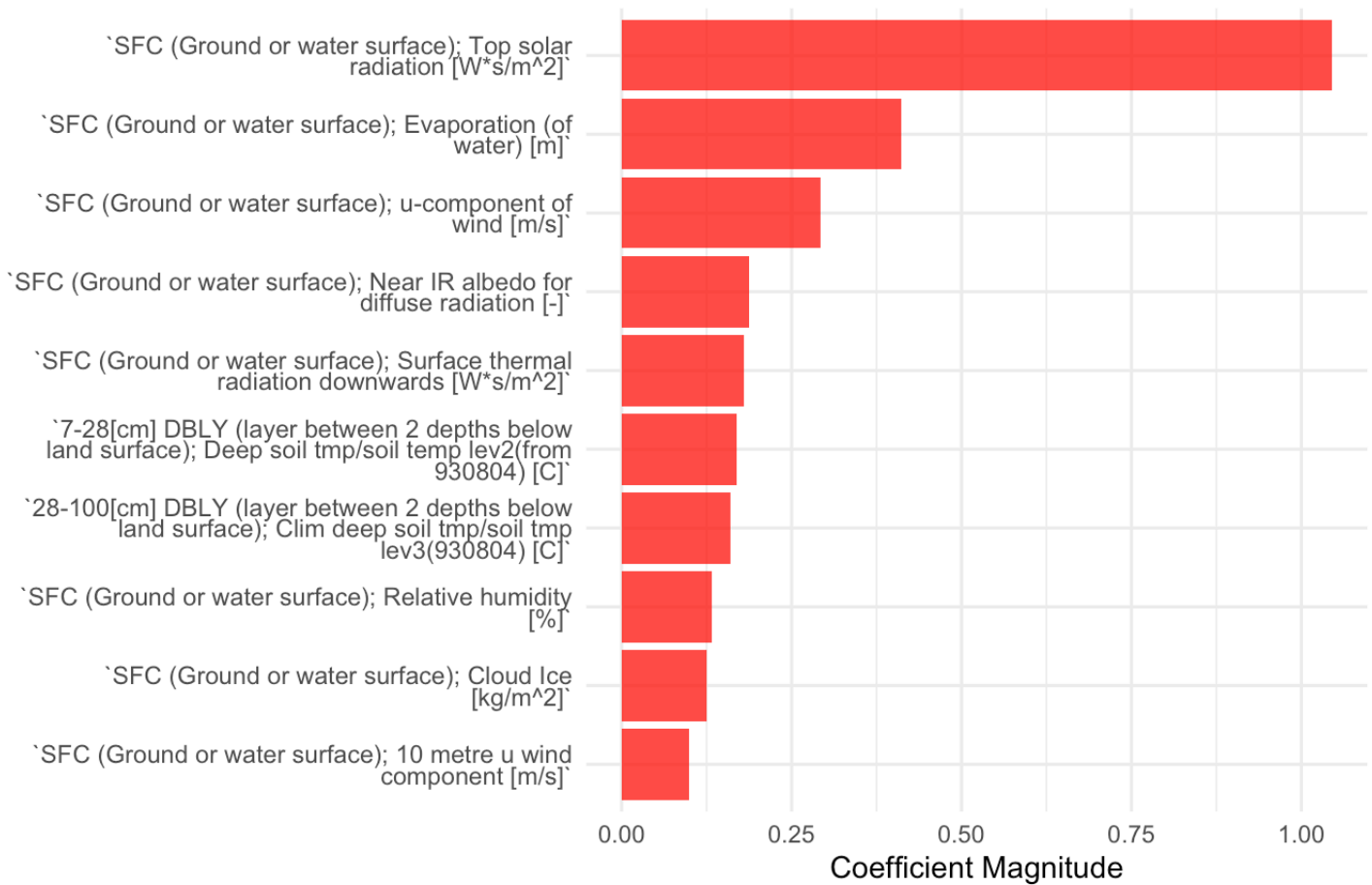
# Top 10 Negative Drivers (Price Suppressors)
plot_negative <- ggplot(df_neg, aes(x = Coefficient, y = Feature_Clean)) +
  geom_bar(stat = "identity", fill = "blue", alpha = 0.8) +
  theme_minimal() +
  labs(
    title = "Top 10 Negative Price Drivers",
    subtitle = "Factors associated with lower electricity prices",
    x = "Coefficient Magnitude",
    y = NULL
  ) +
  theme(
    axis.text.y = element_text(size = 9, lineheight = 0.7),
    plot.title = element_text(face = "bold", size = 12, color = "blue")
  )

print(plot_positive)

```

Top 10 Positive Price Drivers

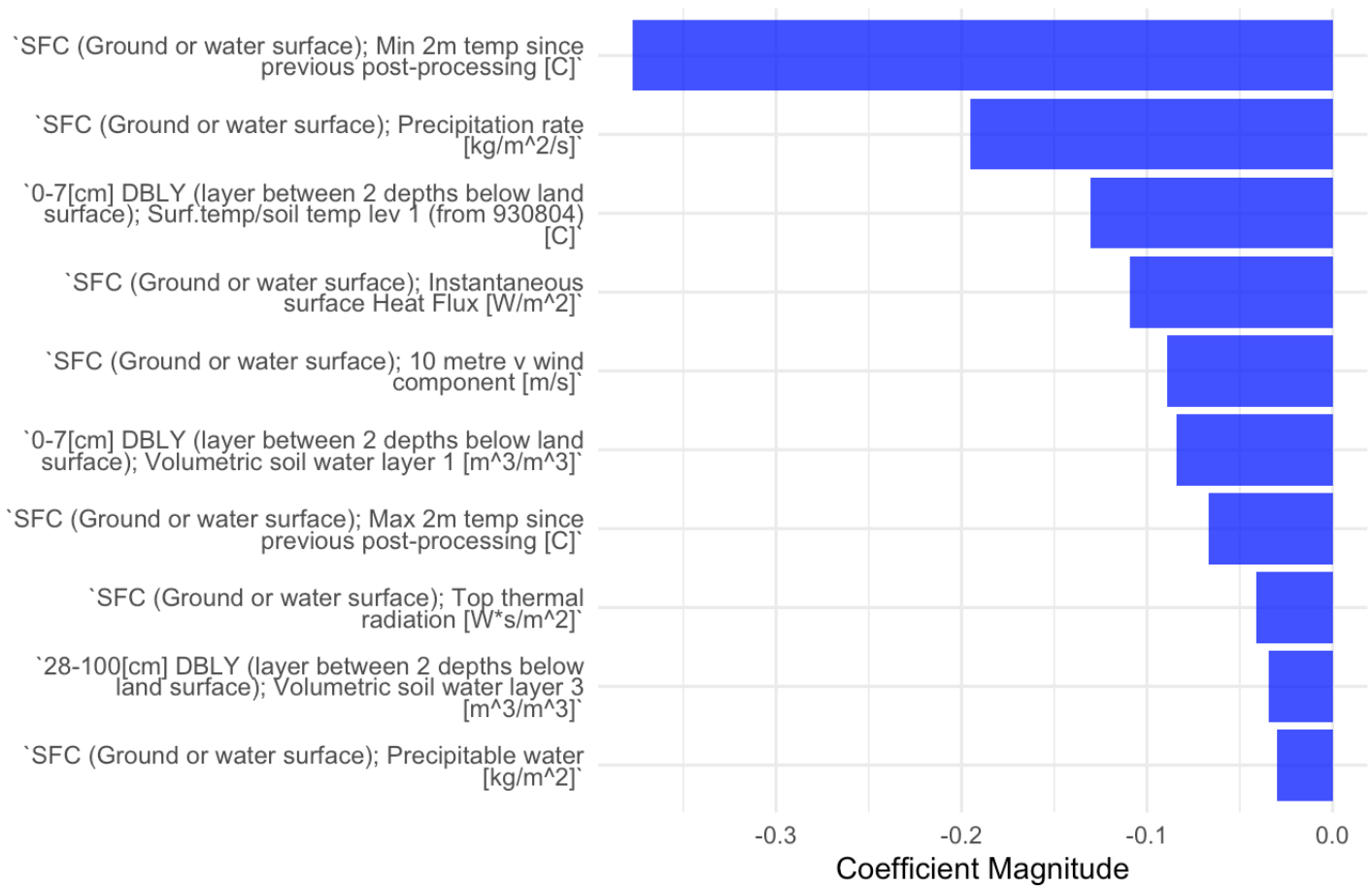
Factors associated with higher electricity prices



```
print(plot_negative)
```

Top 10 Negative Price Drivers

Factors associated with lower electricity prices



```
model_data <- model_data %>%  
  mutate(  
    weekday = weekdays(Datetime),  
    hour = as.numeric(format(Datetime, "%H")),  
  
    time_block = case_when(  
      hour >= 6 & hour < 12 ~ "morning",  
      hour >= 12 & hour < 17 ~ "afternoon",  
      hour >= 17 & hour < 22 ~ "evening",  
      TRUE ~ "night"  
    )  
  )
```

```
pred_log_all <- predict(lasso_model, s = "lambda.min", newx = X_scaled)  
# Convert from $/MWh to $/kWh  
pred_price <- exp(pred_log_all) / 1000  
model_data$pred_price <- pred_price
```

```
weekly_patterns <- model_data %>%
  group_by(weekday, time_block) %>%
  summarise(
    avg_price = mean(pred_price),
    .groups = "drop"
  )
```

```
best_charge <- weekly_patterns %>%
  arrange(avg_price) %>%
  head(5)

best_discharge <- weekly_patterns %>%
  arrange(desc(avg_price)) %>%
  head(5)
```

```
print("Best times to CHARGE:")
```

```
## [1] "Best times to CHARGE:"
```

```
best_charge
```

weekday <chr>	time_block <chr>	avg_price <dbl>
Wednesday	night	0.04705712
Thursday	night	0.04754714
Sunday	night	0.04797689
Monday	night	0.04825194
Tuesday	night	0.04872686

```
5 rows
```

```
print("Best times to DISCHARGE:")
```

```
## [1] "Best times to DISCHARGE:"
```

```
best_discharge
```

weekday <chr>	time_block <chr>	avg_price <dbl>
Thursday	evening	0.10023439

Saturday	evening	0.09748211
Thursday	afternoon	0.09718781
Monday	evening	0.09699753
Wednesday	afternoon	0.09678208

5 rows

```
model_data <- model_data %>%
  mutate(week = format(Datetime, "%Y-%U"))

weekly_recommendations <- model_data %>%
  group_by(week, weekday, time_block) %>%
  summarise(avg_price = mean(pred_price), .groups = "drop")

weekly_summary <- weekly_recommendations %>%
  group_by(week) %>%
  summarise(
    best_charge = paste(head(weekday[order(avg_price)], 2), collapse = ", "),
    best_discharge = paste(head(weekday[order(-avg_price)], 2), collapse = ", ")
  )
```

```
high_usage <- model_data %>%
  group_by(weekday, time_block) %>%
  summarise(avg_usage = mean(Quantity))
```

`summarise()` has grouped output by 'weekday'. You can override using the
`.groups` argument.

```
combined <- weekly_patterns %>%
  left_join(high_usage, by = c("weekday", "time_block"))
```

```
capacity <- 13.5
max_charge_rate <- 5
max_discharge_rate <- 5
efficiency <- 0.9
```

```
model_data <- model_data %>%
  arrange(Datetime) %>%
  mutate(
    future_price = lead(pred_price, 1),
    profit_signal = future_price - pred_price
  )
```



```

model_data <- model_data %>%
  mutate(
    action = case_when(
      pred_price <= quantile(pred_price, 0.25) ~ "charge",
      pred_price >= quantile(pred_price, 0.75) ~ "discharge",
      TRUE ~ "hold"
    )
  )

```

```

soc <- 0
soc_list <- c()

for (i in 1:nrow(model_data)) {

  if (model_data$action[i] == "charge") {
    soc <- min(capacity, soc + max_charge_rate)
  }

  else if (model_data$action[i] == "discharge") {
    soc <- max(0, soc - max_discharge_rate)
  }

  soc_list[i] <- soc
}

model_data$soc <- soc_list

```

```

weekly_strategy <- model_data %>%
  group_by(week, weekday, time_block) %>%
  summarise(
    charge_freq = mean(action == "charge"),
    discharge_freq = mean(action == "discharge"),
    .groups = "drop"
  )

```

```

weekly_recommendations <- weekly_strategy %>%
  group_by(week) %>%
  summarise(
    best_charge = paste(
      head(paste(weekday, time_block)[order(-charge_freq)], 2),
      collapse = ", "
    ),
    best_discharge = paste(
      head(paste(weekday, time_block)[order(-discharge_freq)], 2),
      collapse = ", "
    )
  )

```

```
print("Weekly Recommendations:")
```

```
## [1] "Weekly Recommendations:"
```

```
head(weekly_recommendations, 10)
```

week <chr>	best_charge <chr>	best_discharge <chr>
2022-21	Saturday night, Friday night	Friday evening, Saturday evening
2022-22	Sunday night, Monday night	Friday evening, Saturday evening
2022-23	Tuesday night, Monday night	Friday afternoon, Friday evening
2022-24	Monday night, Thursday night	Friday afternoon, Friday evening
2022-25	Saturday night, Tuesday night	Friday afternoon, Friday evening
2022-26	Sunday night, Friday afternoon	Friday evening, Monday afternoon
2022-27	Monday night, Wednesday night	Friday evening, Monday evening
2022-28	Friday afternoon, Friday evening	Monday afternoon, Monday evening
2022-29	Friday afternoon, Friday evening	Monday evening, Saturday evening
2022-30	Friday afternoon, Friday evening	Friday evening, Monday evening
1-10 of 10 rows		

```

heatmap_data <- model_data %>%
  group_by(weekday, time_block) %>%
  summarise(avg_price = mean(pred_price), .groups = "drop") %>%
  mutate(
    weekday = factor(weekday, levels = c("Monday", "Tuesday", "Wednesday",
                                           "Thursday", "Friday", "Saturday", "Sunda
y")),
    time_block = factor(time_block, levels = c("morning", "afternoon", "evening", "ni
ght"))
  )

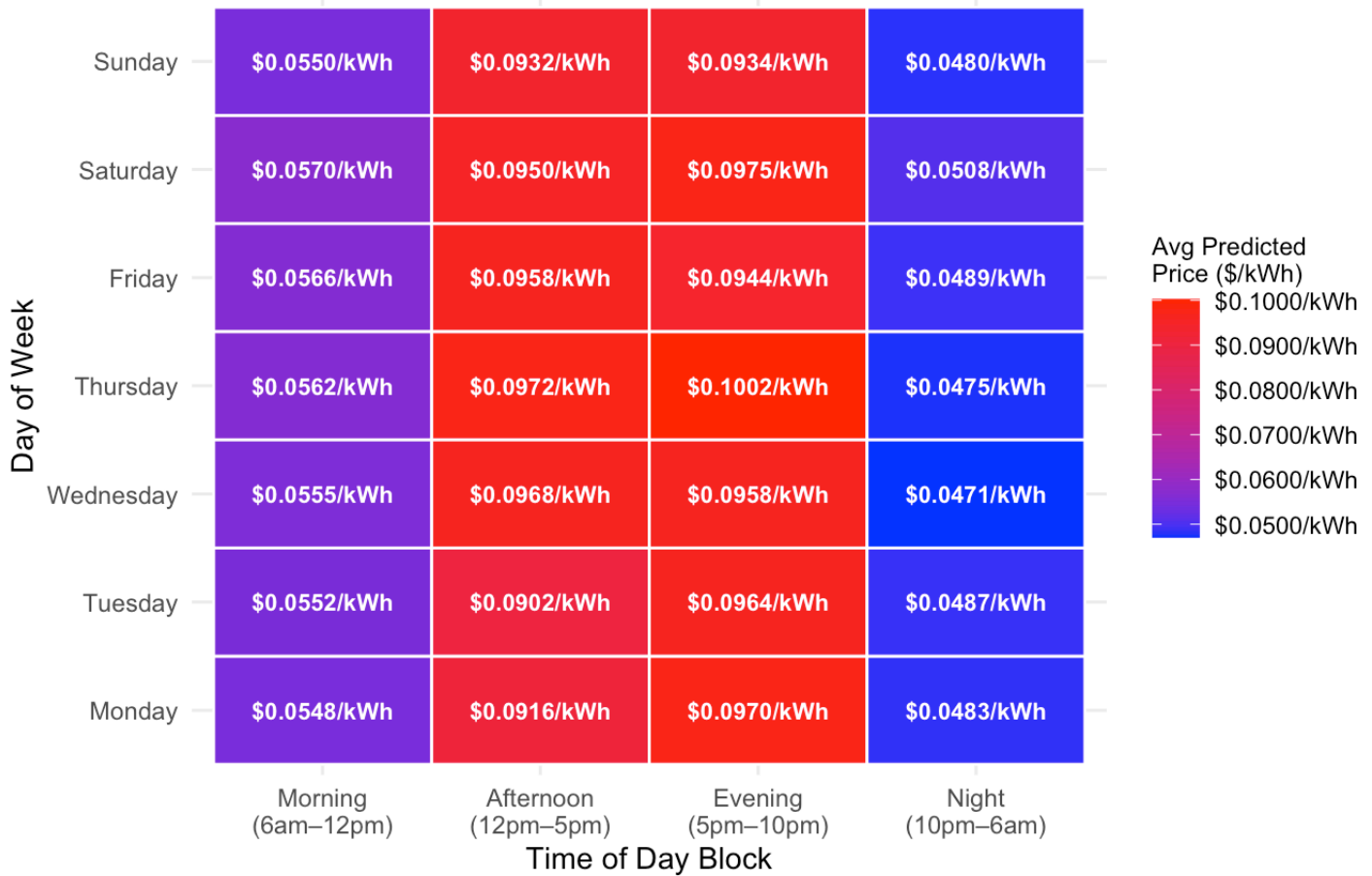
ggplot(heatmap_data, aes(x = time_block, y = weekday, fill = avg_price)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = scales::dollar(avg_price, prefix = "$", suffix = "/kWh",
                                       accuracy = 0.0001)),
            size = 3, color = "white", fontface = "bold") +
  scale_fill_gradient(
    low = "blue", high = "red",
    labels = scales::dollar_format(prefix = "$", suffix = "/kWh", accuracy = 0.0001),
    name = "Avg Predicted\nPrice ($/kWh)"
  ) +
  scale_x_discrete(labels = c("morning" = "Morning\n(6am–12pm)",
                              "afternoon" = "Afternoon\n(12pm–5pm)",
                              "evening" = "Evening\n(5pm–10pm)",
                              "night" = "Night\n(10pm–6am)")) +

  theme_minimal() +
  theme(
    axis.text = element_text(size = 9),
    plot.subtitle = element_text(size = 9, color = "gray40"),
    plot.caption = element_text(size = 8, color = "gray50"),
    legend.title = element_text(size = 9)
  ) +
  labs(
    title = "Average Predicted Electricity Price by Day and Time (May 2022 – Dec 202
2)",
    subtitle = "Blue = low price → best time to charge | Red = high price → best time
to discharge",
    x = "Time of Day Block",
    y = "Day of Week"
  )

```

Average Predicted Electricity Price by Day and Time (May 2022 – Dec 2022)

Blue = low price → best time to charge | Red = high price → best time to discharge



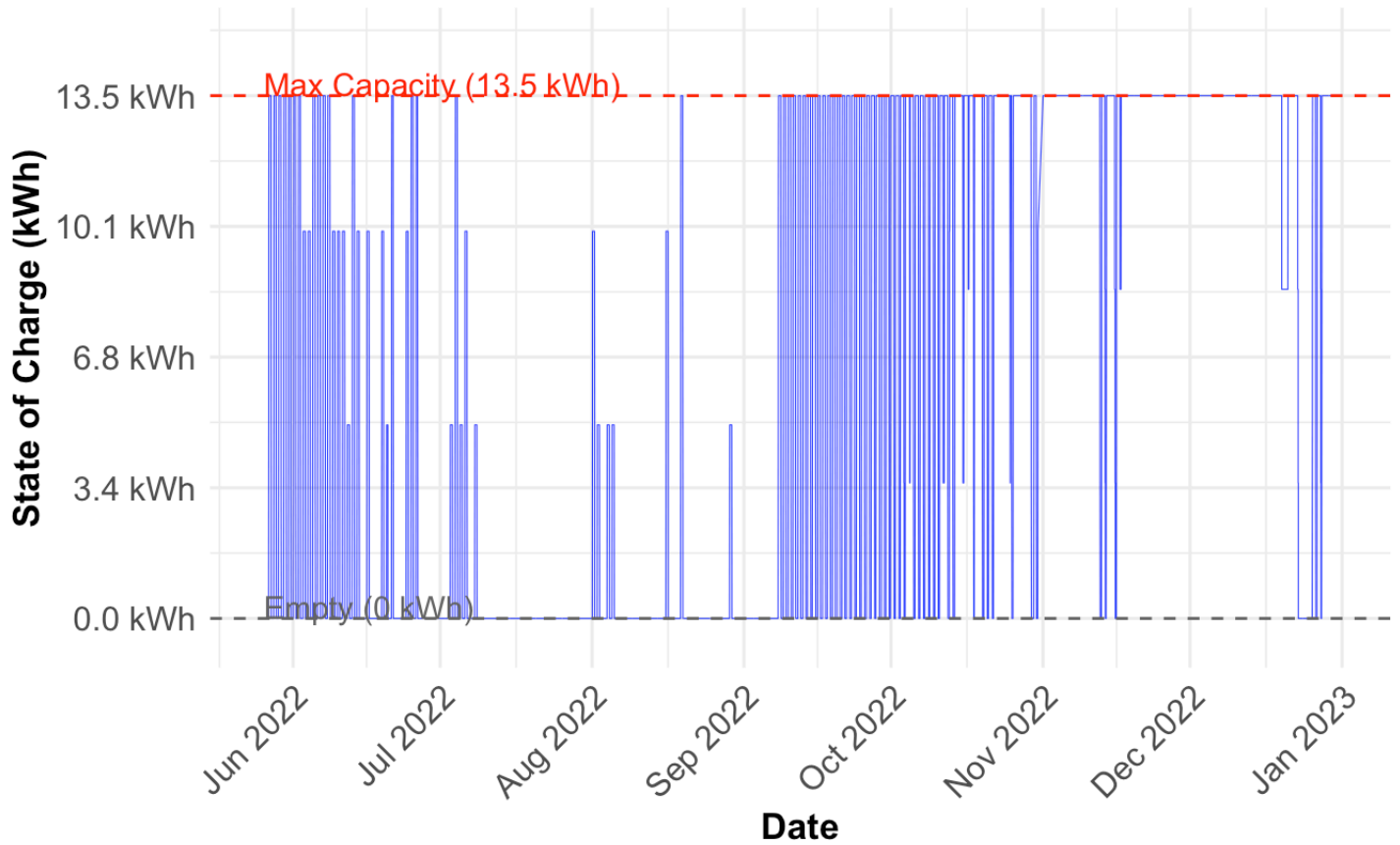
```

ggplot(model_data, aes(x = Datetime, y = soc)) +
  geom_line(color = "blue", linewidth = 0.2, alpha = 0.85) +
  geom_hline(yintercept = capacity, linetype = "dashed", color = "red", linewidth =
0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40", linewidth = 0.5)
+
  annotate("text", x = min(model_data$Datetime), y = capacity + 0.3,
          label = paste0("Max Capacity (", capacity, " kWh)",
          hjust = 0, size = 4, color = "red") +
  annotate("text", x = min(model_data$Datetime), y = 0 + 0.3,
          label = "Empty (0 kWh)", hjust = 0, size = 4, color = "gray40") +
  scale_y_continuous(
    limits = c(-0.5, capacity + 1.5),
    breaks = seq(0, capacity, by = capacity / 4),
    labels = scales::comma_format(suffix = " kWh")
  ) +
  scale_x_datetime(date_breaks = "1 month", date_labels = "%b %Y") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(size = 12, angle = 45, hjust = 1),
    axis.text.y = element_text(size = 12),
    axis.title.x = element_text(size = 13, face = "bold"),
    axis.title.y = element_text(size = 13, face = "bold"),
    plot.title = element_text(size = 14, face = "bold"),
    plot.subtitle = element_text(size = 11, color = "gray40"),
    plot.caption = element_text(size = 10, color = "gray50")
  ) +
  labs(
    title = "Battery State of Charge Over Time (May 2022 – December 2022)",
    subtitle = paste0("Tesla Powerwall | Capacity: ", capacity, " kWh | ",
                      "Charge rate: ", max_charge_rate, " kW | ",
                      "Discharge rate: ", max_discharge_rate, " kW | ",
                      "Efficiency: ", efficiency * 100, "%"),
    x = "Date",
    y = "State of Charge (kWh)",
    caption = "Charge when price <= 25th percentile; Discharge when price >= 75th per
centile"
  )

```

Battery State of Charge Over Time (May 2022 – December 2022)

Tesla Powerwall | Capacity: 13.5 kWh | Charge rate: 5 kW | Discharge rate: 5 kW | Efficiency: 90%



Charge when price $\leq$ 25th percentile; Discharge when price $\geq$ 75th percentile

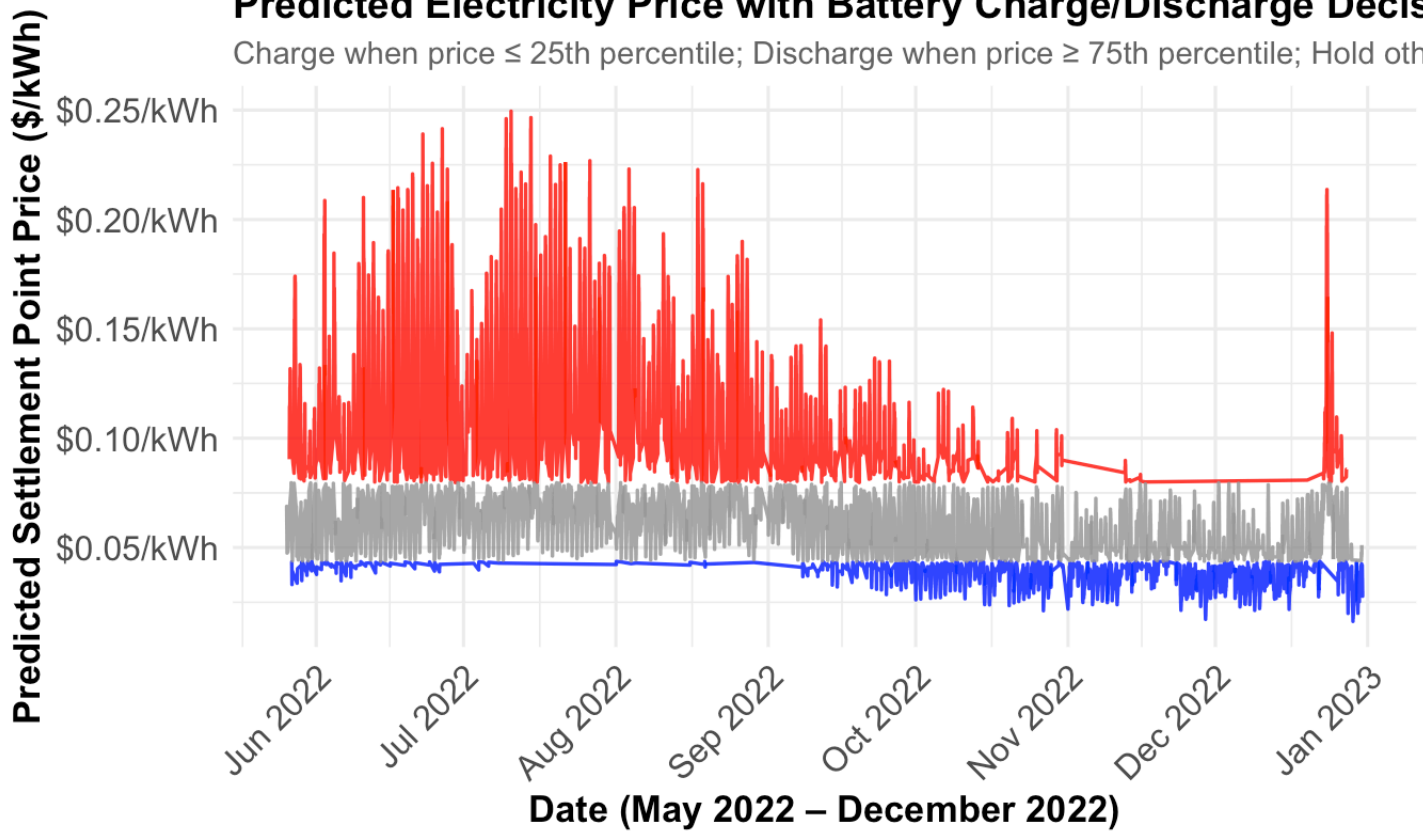
```

ggplot(model_data, aes(x = Datetime, y = pred_price, color = action)) +
  geom_line(linewidth = 0.6, alpha = 0.85) +
  scale_color_manual(
    values = c("charge" = "blue", "discharge" = "red", "hold" = "gray60"),
    labels = c("charge" = "Charge (low price)",
               "discharge" = "Discharge (high price)",
               "hold" = "Hold (neutral)")
  ) +
  scale_y_continuous(labels = scales::dollar_format(prefix = "$", suffix = "/kWh")) +
  scale_x_datetime(date_breaks = "1 month", date_labels = "%b %Y") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(size = 12, angle = 45, hjust = 1),
    axis.text.y = element_text(size = 12),
    axis.title.x = element_text(size = 13, face = "bold"),
    axis.title.y = element_text(size = 13, face = "bold"),
    plot.title = element_text(size = 14, face = "bold"),
    plot.subtitle = element_text(size = 11, color = "gray40"),
    plot.caption = element_text(size = 10, color = "gray50"),
    legend.position = "bottom",
    legend.text = element_text(size = 10),
    legend.title = element_text(size = 11)
  ) +
  labs(
    title = "Predicted Electricity Price with Battery Charge/Discharge Decisions",
    subtitle = "Charge when price  $\leq$  25th percentile; Discharge when price  $\geq$  75th percentile; Hold otherwise",
    x = "Date (May 2022 – December 2022)",
    y = "Predicted Settlement Point Price ($/kWh)",
    color = "Battery Action",
    caption = "Prices predicted using LASSO model; converted from $/MWh to $/kWh"
  )

```

Predicted Electricity Price with Battery Charge/Discharge Decis

Charge when price $\leq$ 25th percentile; Discharge when price $\geq$ 75th percentile; Hold other



Battery Action — Charge (low price) — Discharge (high price) — Hold (neutral)

Prices predicted using LASSO model; converted from \$/MWh to \$/kWh

```
profit <- sum(
  ifelse(model_data$action == "discharge", model_data$pred_price, 0) -
  ifelse(model_data$action == "charge", model_data$pred_price, 0)
)

print(paste("Estimated Profit from Strategy:", round(profit, 2)))
```

```
## [1] "Estimated Profit from Strategy: 229.36"
```

```
rmse <- sqrt(mean((pred - y_test_original)^2))
mae <- mean(abs(pred - y_test_original))

rmse
```

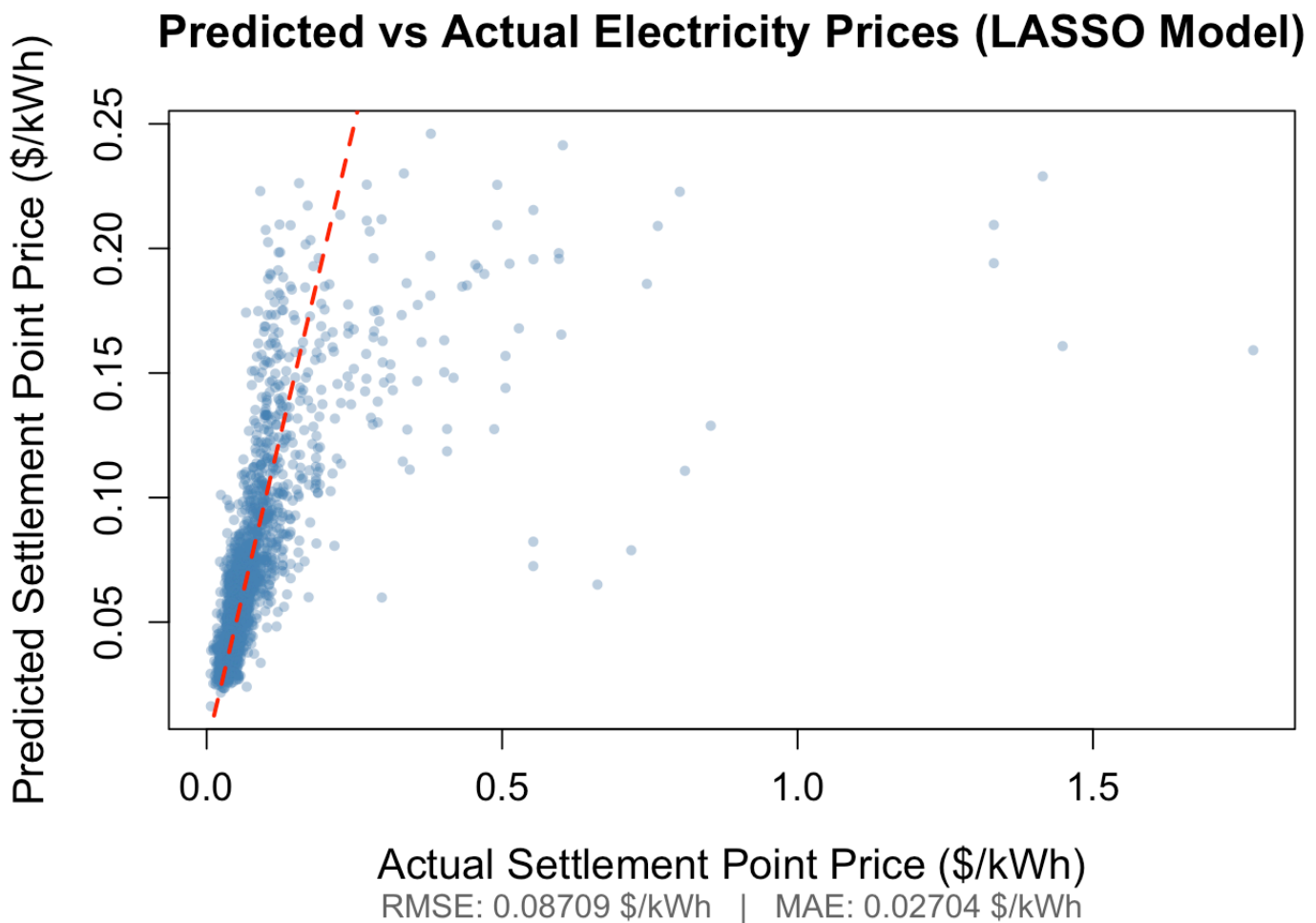
```
## [1] 0.08709471
```

```
mae
```



```
## [1] 0.02704294
```

```
plot(y_test_original, pred,  
     xlab = "Actual Settlement Point Price ($/kWh)",  
     ylab = "Predicted Settlement Point Price ($/kWh)",  
     main = "Predicted vs Actual Electricity Prices (LASSO Model)",  
     sub = paste0("RMSE: ", round(rmse, 5), " $/kWh | MAE: ", round(mae, 5), "  
$/kWh"),  
     pch = 16,  
     col = adjustcolor("steelblue", alpha.f = 0.4),  
     cex = 0.7,  
     cex.main = 1.4,  
     cex.lab = 1.3,  
     cex.axis = 1.2,  
     cex.sub = 1.0,  
     col.sub = "gray40")  
  
abline(0, 1, col = "red", lwd = 2, lty = 2)
```



```

plot(y_test_original,
     type = "l",
     col = "red",
     lwd = 1.5,
     xlab = "Observation Index (Test Set)",
     ylab = "Settlement Point Price ($/kWh)",
     main = "Predicted vs Actual Electricity Prices (LASSO Model)",
     cex.main = 1.4,    # title size
     cex.lab  = 1.3,    # axis title size
     cex.axis = 1.2,    # axis tick label size
     cex.sub  = 1.1,
     col.sub  = "gray40",
     ylim = range(c(y_test_original, pred), na.rm = TRUE))

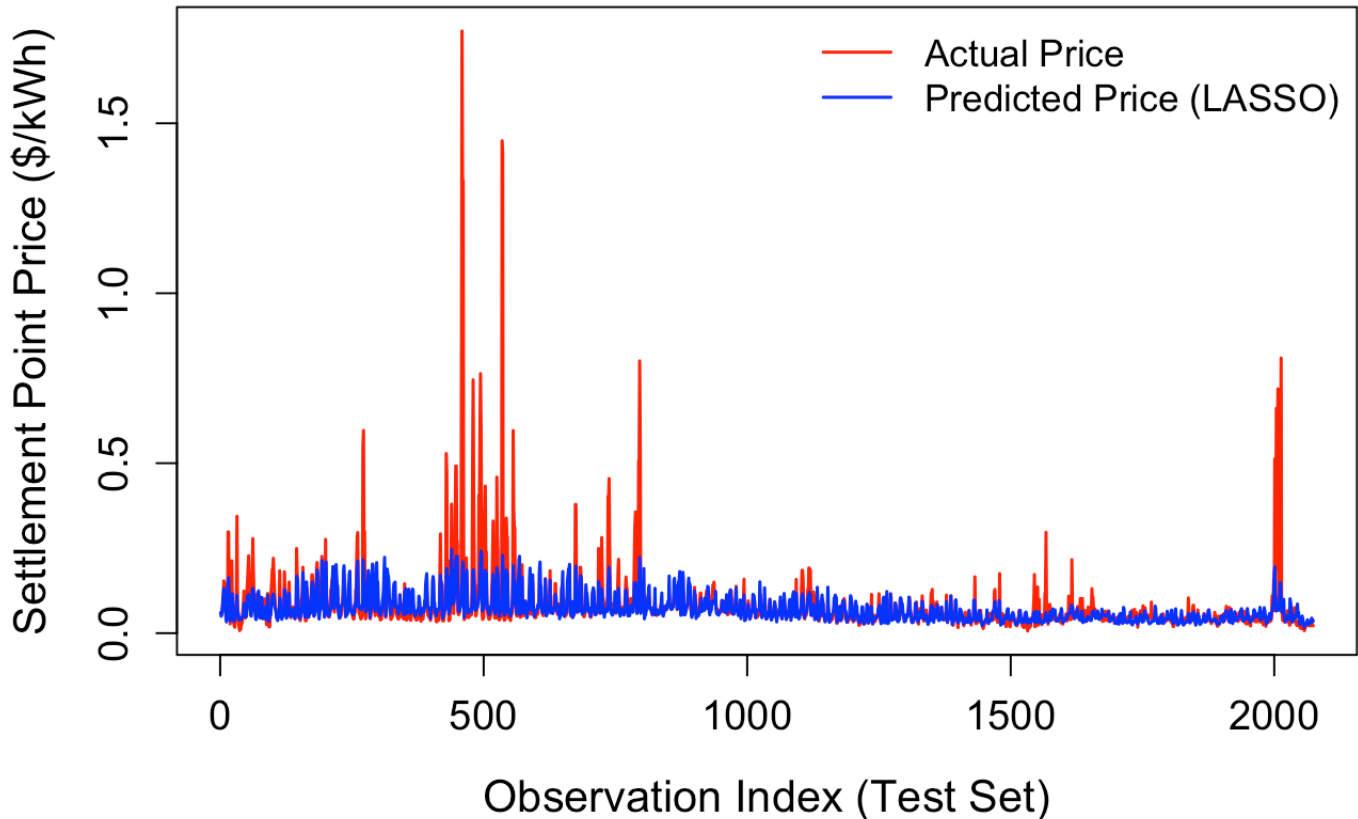
lines(pred, col = "blue", lwd = 1.5, lty = 1)

legend("topright",
      legend = c("Actual Price", "Predicted Price (LASSO)"),
      col     = c("red", "blue"),
      lty     = c(1, 1),
      lwd     = 1.5,
      bty     = "n",
      cex     = 1.1)    # legend text size

mtext(paste0("RMSE: ", round(rmse, 5), " $/kWh   |   MAE: ", round(mae, 5), " $/kW
h"),
      side = 1, line = 4, cex = 1.0, col = "gray40")

```

Predicted vs Actual Electricity Prices (LASSO Model)



```
baseline_pred <- mean(y_train)
```

```
baseline_rmse <- sqrt(mean((baseline_pred - y_test_original)^2))
```

```
baseline_rmse
```

```
## [1] 4.042234
```